Question 1: A database is decided for a large construction enterprise to support its resources management. Such an enterprise has a set of Departments and Projects. Each department has a set of Equipment. Equipment's are assigned to projects for a specific period of time. •No equipment is assigned to more than one project at a time. •An equipment could be idle for a given period of time. •The database is to contain the following information: o Department data together, with data concerning the equipment of each department. •Project's data together with necessary data of all equipment assigned to each project (start and finish dates of assigning an equipment to a project). •For each department: department number (unique), name (unique), and budget. •For each equipment: equipment number (unique), name, location , project start date , project finish date . •For each project: project number (unique), name, location, project start date and project finish date.

Question 1 – ER Modeling Solution

Solution 1: Using M:M Relationship (Direct Assignment)

Note: if repetition is not needed for M:M relationship

Entity Types

Department

- **Department_Number** (Primary Key)
- Name (Unique)
- Budget

Equipment

- **Equipment_Number** (Primary Key)
- Name
- Location

Project

- **Project_Number** (Primary Key)
- Name

- Location

Relationships

Owns

- Between **Department** and **Equipment**
- Cardinality: **1 : M**
- One department owns many equipment.
- Each equipment belongs to exactly one department.

Assigned_To

- Between **Equipment** and **Project**
- Cardinality: **M : M**
- Relationship Attributes:
    - Assignment_Start_Date
    - Assignment_Finish_Date

Solution 2: Resolving M:M Relationship Using Transaction Entity

Note: if repetition is needed for M:M relationship

Entity Types

Department

- **Department_Number** (Primary Key)
- Name (Unique)
- Budget

Equipment

- **Equipment_Number** (Primary Key)
- Name
- Location
- Department_Number (Foreign Key)

Project

- **Project_Number** (Primary Key)
- Name
- Location

Transaction

- **Transaction_Number** (Primary Key)
- Assignment_Start_Date
- Assignment_Finish_Date
- Equipment_Number (Foreign Key)
- Project_Number (Foreign Key)

Relationships

Owns

- Between **Department** and **Equipment**
- Cardinality: **1 : M**

Equipment_Assignment

- Between **Equipment** and **Transaction**
- Cardinality: **1 : M**
- One equipment can be involved in multiple transactions over time.

Project_Assignment

- Between **Project** and **Transaction**
- Cardinality: **1 : M**
- One project can have many equipment assignments.

Note: Check if repetition is needed or not for any M:M relationship

Question2: A database contains information concerning sales representatives, sales areas, and products. Each representative is responsible for sales in only one area. Each area may have one or more responsible representatives. Each representative is responsible for sales of one or more products, and each product has one or more responsible representatives. A sales commission is assigned to each representative. The commission rate is determined according to the sales area, in other words, for each area there is a certain fixed commission rate for all responsible representatives of that area. The database is to contain the following information: •For each representative: representative number (unique), representative name (unique), and sales commission. •For each sales area: area number (unique), and location. •For each product: product number (unique), price,product name and quantity sold by each representative in each area.

ER Modeling Solution – Sales Representatives Database

Solution 1: Using M:M Relationship (Direct Modeling)

Note: if repetition is not needed for M:M relationship

Entity Types

Sales_Representative

- **Representative_Number** (Primary Key)
- Representative_Name (Unique)
- Sales_Commission (dervied attribute)

Sales_Area

- **Area_Number** (Primary Key)
- Location
- Commission_Rate

Product

- **Product_Number** (Primary Key)

- Product_Name
- Price

Relationships

Assigned_To

- Between **Sales_Representative** and **Sales_Area**
- Cardinality: **M : 1**
- Each representative works in exactly one sales area.
- Each sales area may have one or more representatives.

Sells

- Between **Sales_Representative** and **Product**
- Cardinality: **M : M**
- Relationship Attribute:
    - Quantity_Sold

Solution 2: Resolving M:M Relationship Using Transaction Entity

Note: if repetition is needed for M:M relationship

Entity Types

Sales_Representative

- **Representative_Number** (Primary Key)
- Representative_Name (Unique)
- Sales_Commission (dervied attribute)
- Area_Number (Foreign Key)

Sales_Area

- **Area_Number** (Primary Key)
- Location
- Commission_Rate

Product

- **Product_Number** (Primary Key)
- Product_Name
- Price

Transaction

- **Transaction_Number** (Primary Key)
- Sub_Quantity_Sold
- Representative_Number (Foreign Key)
- Product_Number (Foreign Key)

Relationships

Works_In

- Between **Sales_Area** and **Sales_Representative**
- Cardinality: **1 : M**

Representative_Transaction

- Between **Sales_Representative** and **Transaction**
- Cardinality: **1 : M**
- One representative can have many sales transactions.

Product_Transaction

- Between **Product** and **Transaction**
- Cardinality: **1 : M**
- One product can appear in many transactions.

Note: Check if repetition is needed or not for any M:M relationship

Question3: A database contains information concerning sales representatives, sales areas, and products. Each representative is responsible for sales in more than one area. Each area may have one or more responsible representatives. Each representative is responsible for sales of one or more products, and each product has one or more responsible representatives. A sales commission is assigned to each representative. The commission rate is determined according to the sales area, in other words, for each area there is a certain fixed commission rate for all responsible representatives of that area. The database is to contain the following information: •For each representative: representative number (unique), representative name (unique), and sales commission. •For each sales area: area number (unique), and location. •For each product: product number (unique), price,product name and quantity sold by each representative in each area.

ER Modeling Solution – Sales Representatives Database

Solution 1: Using M:M Relationship (Direct Modeling)

Note: if repetition is not needed for M:M relationship

Entity Types

Sales_Representative

- **Representative_Number** (Primary Key)
- Representative_Name (Unique)
- Sales_Commission (dervied attribute)

Sales_Area

- **Area_Number** (Primary Key)

- Location
- Commission_Rate

Product

- **Product_Number** (Primary Key)
- Product_Name
- Price

Relationships

Sells

- Between **Sales_Representative** and **Product** and **Sales_Area**
- Cardinality: **M: M:M**
- Relationship Attribute:
    - Quantity_Sold

Solution 2: Resolving M:M Relationship Using Transaction Entity

Note: if repetition is needed for M:M relationship

Entity Types

Sales_Representative

- **Representative_Number** (Primary Key)
- Representative_Name (Unique)
- Sales_Commission (dervied attribute)
- Area_Number (Foreign Key)

Sales_Area

- **Area_Number** (Primary Key)
- Location
- Commission_Rate

Product

- **Product_Number** (Primary Key)
- Product_Name
- Price

Transaction

- **Transaction_Number** (Primary Key)
- Sub_Quantity_Sold
- Representative_Number (Foreign Key)
- Product_Number (Foreign Key)
- SalesArea_Number (Foreign Key)

Relationships

Sales_Area _Transaction

- Between **Sales_Area** and **Transaction**
- Cardinality: **1 : M**

Representative_Transaction

- Between **Sales_Representative** and **Transaction**
- Cardinality: **1 : M**
- One representative can have many sales transactions.

Product_Transaction

- Between **Product** and **Transaction**

- Cardinality: **1 : M**
- One product can appear in many transactions.

Note: Check if repetition is needed or not for any M:M relationship

Question4: A database is decided for scheduling the coming football tournament of World Cup in France. The participating teams are divided into groups each of 8 teams. The designed database must contain data about:

Teams described by country, coach, and the score of these team in each match. Groups described by group number, teams in that groups and the group supervisor name. Matches described by match id (unique) , the teams, referee , court number, court city and date. referees described by referee_id (unique key),name .Players described by teams, player number, player name, player position in the team, number of goals for each player in each match there is a home team and an away (visiting) team.

ER Modeling Solution – World Cup Tournament Database

Solution 1: Using M:M Relationship (Direct Modeling)

Note: if repetition is not needed for M:M relationship

Entity Types

Group

- **Group_Number** (Primary Key)
- Supervisor_Name

Team

- **Team_ID** (Primary Key)
- Country
- Coach
- Group_Number (Foreign Key)

Match

- **Match_ID** (Primary Key)
- Court_Number
- Court_City
- Match_Date
- Referee_ID (Foreign Key)
- TeamVisitor_ID(Foreign Key)
- TeamHome_ID(Foreign Key)
- Score_visit
- Score_home

Referee

- **Referee_ID** (Primary Key)
- Name

Player

- **Player_Number** (Primary Key)
- Player_Name
- Player_Position
- Team_ID (Foreign Key)

Relationships

Group_Team

- Between **Group** and **Team**
- Cardinality: **1 : M**

Plays_visit

- Between **Team** and **Match**
- Cardinality: 1 **: M**
- Relationship Attribute:

    - TeamVisotor_Score

Plays_home

- Between **Team** and **Match**
- Cardinality: 1 **: M**
- Relationship Attribute:
    - TeamHome_Score

Officiates

- Between **Referee** and **Match**
- Cardinality: **1 : M**
- Each match has one referee.
- A referee can officiate many matches.

Belongs_To

- Between **Team** and **Player**
- Cardinality: **1 : M**
- Each player belongs to one team.
- Each team has many players.

Participates

- Between **Player** and **Match**
- Cardinality: **M : M**
- Relationship Attribute:
    - Goals_Scored

Solution 2: Resolving M:M Relationship Using Transaction Entity

Note: if repetition is needed for M:M relationship

Entity Types

Group

- **Group_Number** (Primary Key)
- Supervisor_Name

Team

- **Team_ID** (Primary Key)
- Country
- Coach
- Group_Number (Foreign Key)

Match

- **Match_ID** (Primary Key)
- Court_Number
- Court_City
- Match_Date
- Referee_ID (Foreign Key)
- TeamVisitor_ID(Foreign Key)
- TeamHome_ID(Foreign Key)
- Score_visit
- Score_home

Referee

- **Referee_ID** (Primary Key)
- Name

Player

- **Player_Number** (Primary Key)
- Player_Name
- Player_Position
- Team_ID (Foreign Key)

Transaction

- **Transaction_Number** (Primary Key)
- Player_Number(Foreign Key)
- Match_ID(Foreign Key)

- Goals_Scored

Relationships

Group_Team

- Between **Group** and **Team**
- Cardinality: **1 : M**

Plays_visit

- Between **Team** and **Match**
- Cardinality: 1 **: M**
- Relationship Attribute:
    - TeamVisotor_Score

Plays_home

- Between **Team** and **Match**
- Cardinality: **1 : M**
- Relationship Attribute:
    - TeamHome_Score

Officiates

- Between **Referee** and **Match**
- Cardinality: **1 : M**
- Each match has one referee.
- A referee can officiate many matches.

Belongs_To

- Between **Team** and **Player**
- Cardinality: **1 : M**
- Each player belongs to one team.
- Each team has many players.

Player_Transaction

- Between **Player** and **Transaction**

- Cardinality: **1 : M**
- Each player belongs to one transaction.
- Each transaction has many players.

Match_Transaction

- Between **Match** and **Transaction**
- Cardinality: **1 : M**
- Each Match belongs to one transaction.
- Each transaction has many Matches.

Note: Check if repetition is needed or not for any M:M relationship

Question4: Consider the following set of requirements for EGYPTAIR database which contains information about the airline flight system of that national company: • Each FLIGHT is identified by a flight number ,has flight code and date. Each flight consists of one or more flight legs. A flight leg is a nonstop portion of a flight. • Each FLIGHT LEG has number (1, 2, 3, etc.), departure airport, scheduled departure time (day-hour-min), arrival airport, scheduled arrival time (dayhour-min), and airplane. The leg number is unique for flight legs of the same flight. • Each AIRPORT has code (unique), name, and city • Each AIRPLANE TYPE has type name (unique), company, and maximum number of seats. Each airplane type can land only on certain airports. There is a set of airplanes for each airplane type. • Each AIRPLANE has ID number (unique for airplanes of the same type), and technical status (in service, out of service, in maintenance, ... etc.).

ER Modeling Solution – EGYPTAIR Flight Database

Solution 1: Using M:M Relationships (Direct Modeling)

Note: if repetition is not needed for M:M relationship

Entity Types

Flight

- **Flight_Number** (Primary Key)
- Flight_Code
- Flight_Date

Flight_Leg

- **Leg_Number** (Partial Key)
- Departure_Time
- Arrival_Time

Airport

- **Airport_Code** (Primary Key)
- Name
- City

Airplane_Type

- **Type_Name** (Primary Key)
- Company
- Max_Number_of_Seats

Airplane

- **Airplane_ID** (Primary Key)
- Technical_Status

Relationships

Consists_Of

- Between **Flight** and **Flight_Leg**
- Cardinality: **1 : M**
- Each flight consists of one or more flight legs.
- Leg number is unique within the same flight.

Departs_From

- Between **Flight_Leg** and **Airport**
- Cardinality: **M : 1**

Arrives_At

- Between **Flight_Leg** and **Airport**
- Cardinality: **M : 1**

Uses

- Between **Flight_Leg** and **Airplane**
- Cardinality: **M : 1**
- Each leg uses one airplane.
- An airplane can be used for many flight legs.

Of_Type

- Between **Airplane** and **Airplane_Type**
- Cardinality: **M : 1**

Can_Land_On

- Between **Airplane_Type** and **Airport**
- Cardinality: **M : M**
- Each airplane type can land only on certain airports.
- Each airport can accept multiple airplane types.

Solution 2: Resolving M:M Relationship Using Transaction Entity

Note: if repetition is needed for M:M relationship

Entity Types

Flight

- **Flight_Number** (Primary Key)
- Flight_Code
- Flight_Date

Flight_Leg

- **Leg_Number** (Partial Key)
- Departure_Time
- Arrival_Time
- Flight_Number (Foreign Key)
- Departure_Airport (Foreign Key)
- Arrival_Airport (Foreign Key)
- Airplane_ID (Foreign Key)

Airport

- **Airport_Code** (Primary Key)
- Name
- City

Airplane_Type

- **Type_Name** (Primary Key)
- Company

- Max_Number_of_Seats

Airplane

- **Airplane_ID** (Primary Key)
- Technical_Status
- Type_Name (Foreign Key)

Transaction

- **Transaction_Number** (Primary Key)
- Type_Name (Foreign Key)
- Airport_Code (Foreign Key)

Relationships

Flight_Leg_Assignment

- Between **Flight** and **Flight_Leg**
- Cardinality: **1 : M**

Leg_Departure

- Between **Airport** and **Flight_Leg**
- Cardinality: **1 : M**

Leg_Arrival

- Between **Airport** and **Flight_Leg**

- Cardinality: **1 : M**

Airplane_Assignment

- Between **Airplane** and **Flight_Leg**
- Cardinality: **1 : M**

Airplane_Type_Assignment

- Between **Airplane_Type** and **Airplane**
- Cardinality: **1 : M**

Landing_Permission

- Between **Airplane_Type** and **Transaction**
- Cardinality: **1 : M**

Airport_Permission

- Between **Airport** and **Transaction**
- Cardinality: **1 : M**

Note: Check if repetition is needed or not for any M:M relationship

Question 5: A database contains information about the internal education system of a large industrial company. The company in question maintains an education department whose function is to run several training courses for the employees of the company; each course is offered at a number of different locations within the organization, and the database contains details both of offerings already given and of offerings scheduled to be given in future. The database contains the following information:- For each course: course number (unique), course title, details of all immediate prerequisite course, and details of all offerings For each offering, of a given course; offering number, date, location, details of the teacher, and details of all students For each teacher of a given

offering; employee number (unique), and name For each student of a given offering; employee number (unique), name, and grade The following constraints apply:

- Each course has only one prerequisite course, while one prerequisite course may be associated with many courses.

-Each course may have multiple offerings, while each offering belongs to one course.

-Each offering may include many students, and each student may be enrolled in many offerings.

ER Modeling Solution – Internal Education System Database

Solution 1: Using Relationships Using M:M Relationships (Direct Modeling)

Note: if repetition is not needed for M:M relationship

Entity Types

Course

- **Course_Number** (Primary Key)
- Course_Title
- Prerequisite_Course_Number (Foreign Key → Course)
- Teacher__Number (Foreign Key)

Offering

- **Offering_Number** (Primary Key)
- Offering_Date
- Location
- Course_Number (Foreign Key)

Teacher

- **Employee_Number** (Primary Key)
- Name

Student

- **Student_Number** (Primary Key)
- Name

Relationships

Prerequisite

- Recursive relationship on **Course**
- Cardinality: **1 : M**
- One course may be a prerequisite for many courses.
- Each course has **only one immediate prerequisite** (or none).

Has_Offering

- Between **Course** and **Offering**
- Cardinality: **1 : M**
- Each course has multiple offerings.
- Each offering belongs to exactly one course.

Teaches

- Between **Teacher** and **Course**
- Cardinality: **1 : M**
- Each course is taught by only one teacher.
- A teacher may teach more than one course.

Attends

- Between **Student** and **Offering**
- Cardinality: **M : M**
- Relationship Attribute:
    - Grade

Solution 2: Resolving M:M Relationships Using Transaction Entity

Note: if repetition is needed for M:M relationship

Entity Types

Course

- **Course_Number** (Primary Key)
- Course_Title
- Prerequisite_Course_Number (Foreign Key → Course)
- Teacher__Number (Foreign Key)

Offering

- **Offering_Number** (Primary Key)
- Offering_Date
- Location
- Course_Number (Foreign Key)

Teacher

- **Employee_Number** (Primary Key)
- Name

Student

- **Student_Number** (Primary Key)
- Name

Transaction

- **Transaction_Number** (Primary Key)
- Grade
- Offering_Number (Foreign Key)
- Student_Number (Foreign Key)

Relationships

Course_Prerequisite

- Between **Course** and **Course**
- Cardinality: **1 : M**
- Each course has at most one prerequisite.

Course_Offering

- Between **Course** and **Offering**
- Cardinality: **1 : M**

Course_Teacher

- Between **Teacher** and **Course**
- Cardinality: **1 : M**

Student_Transaction

- Between **Student** and **Transaction**
- Cardinality: **1 : M**

Offering_Transaction

- Between **Offering** and **Transaction**
- Cardinality: **1 : M**

Note: Check if repetition is needed or not for any M:M relationship

Question 6: A database is to contain information about the internal education system of a large industrial company. The company in question maintains an education department whose function is to run several training courses for the employees of the company; each course is offered at a number of different locations within the organization, and the database contains details both of offerings already given and of offerings scheduled to be given in future. The database contains the following information:- For each course: course number (unique), course title, details of all immediate prerequisite course, and details of all offerings For each offering, of a given course; offering number, date, location, details of the teacher, and details of all students For each teacher of a given offering; employee number (unique), and name For each student of a given offering; employee number (unique), name, and grade The following constraints apply:

- Each course has more than one prerequisite course, while one prerequisite course may be associated with many courses.

-Each course may have multiple offerings, while each offering belongs to one course.

-Each offering may include many students, and each student may be enrolled in many offerings.

ER Modeling Solution – Internal Education System Database

Solution 1: Using Relationships Using M:M Relationships (Direct Modeling)

Note: if repetition is not needed for M:M relationship

Entity Types

Course

- **Course_Number** (Primary Key)
- Course_Title
- Teacher_Number (Foreign Key)

Offering

- **Offering_Number** (Primary Key)
- Offering_Date
- Location
- Course_Number (Foreign Key)

Teacher

- **Employee_Number** (Primary Key)
- Name

Student

- **Employee_Number** (Primary Key)
- Name

Relationships

Prerequisite

- Recursive relationship on **Course**

- Cardinality: **M : M**
- One course may be a prerequisite for many courses.
- Each course has **more than one immediate prerequisite**.

Has_Offering

- Between **Course** and **Offering**
- Cardinality: **1 : M**
- Each course has multiple offerings.
- Each offering belongs to exactly one course.

Teaches

- Between **Teacher** and **Course**
- Cardinality: **1 : M**
- Each course is taught by only one teacher.
- A teacher may teach more than one course.

Attends

- Between **Student** and **Offering**
- Cardinality: **M : M**
- Relationship Attribute:
    - Grade

Solution 2: Resolving M:M Relationships Using Transaction Entity

Note: if repetition is needed for M:M relationship

Entity Types

Course

- **Course_Number** (Primary Key)
- Course_Title
- Teacher_Number (Foreign Key)

Offering

- **Offering_Number** (Primary Key)
- Offering_Date
- Location
- Course_Number (Foreign Key)

Teacher

- **Employee_Number** (Primary Key)
- Name

Student

- **Employee_Number** (Primary Key)
- Name

Transaction1

- **Transaction_Number1**(Primary Key)
- Grade
- Offering_Number (Foreign Key)
- Student_Employee_Number (Foreign Key)

Transaction2

- **Transaction_Number2** (Primary Key)
- Main_courseNumber (Foreign Key)
- Prerequisite_courseNumber(Foreign Key)

Relationships

CoursePrerequisite_Transaction2

- Between **Course** and **Transaction2**
- Cardinality: **1 : M**

CourseMain_Transaction2

- Between **Course** and **Transaction2**
- Cardinality: **1 : M**

Course_Offering

- Between **Course** and **Offering**
- Cardinality: **1 : M**

Course_Teacher

- Between **Teacher** and **Course**
- Cardinality: **1 : M**

Student_Transaction1

- Between **Student** and **Transaction1**
- Cardinality: **1 : M**

Offering_Transaction1

- Between **Offering** and **Transaction1**
- Cardinality: **1 : M**

Note: Check if repetition is needed or not for any M:M relationship

Question7: A database is to contain information about the internal education system of a large industrial company. The company in question maintains an education department whose function is to run several training courses for the employees of the company; each course is offered at a number of different locations within the organization, and the database contains details both of offerings already given and of offerings scheduled to be given in future. The database contains the following information:- For each course: course number (unique), course title, details of all immediate prerequisite course, and details of all offerings For each offering, of a given course; offering number, date, location, details of the teacher, and details of all students For each teacher of a given offering; employee number (unique), and name For each student of a given offering; employee number (unique), name, and grade The following constraints apply:

- Each course has only one prerequisite course, while one prerequisite course may be associated with many courses.

-Each course is taught by only one teacher, while a teacher may teach more than one course.

-Each course may have multiple offerings, while each offering belongs to one course.

-Each offering may include many students, and each student may be enrolled in many offerings.

ER Modeling Solution – Internal Education System Database

Solution 1: Using Relationships Using M:M Relationships (Direct Modeling)

Note: if repetition is not needed for M:M relationship

Entity Types

Course

- **Course_Number** (Primary Key)
- Course_Title
- Prerequisite_Course_Number (Foreign Key → Course)

Offering

- **Offering_Number** (Primary Key)
- Offering_Date
- Location
- **Course_Number**(Foreign Key)
- **Teacher_Number**(Foreign Key)

Teacher

- **Employee_Number** (Primary Key)
- Name

Student

- **Student_Number** (Primary Key)
- Name

Relationships

Prerequisite

- Recursive relationship on **Course**

- Cardinality: **1 : M**
- One course may be a prerequisite for many courses.
- Each course has **only one immediate prerequisite** (or none).

Has_Offering

- Between **Course** and **Offering**
- Cardinality: **1 : M**

Teaches

- Between **Teacher** and **Offering**
- Cardinality: **1 : M**

Attends

- Between **Student** and **Offering**
- Cardinality: **M : M**
- Relationship Attribute:
    - Grade

Solution 2: Resolving M:M Relationships Using Transaction Entity

Note: if repetition is needed for M:M relationship

Entity Types

Course

- **Course_Number** (Primary Key)
- Course_Title

- Prerequisite_Course_Number (Foreign Key → Course)

Offering

- **Offering_Number** (Primary Key)
- Offering_Date
- Location
- Course_Number (Foreign Key)
- Teacher_Number (Foreign Key)

Teacher

- **Employee_Number** (Primary Key)
- Name

Student

- **Student_Number** (Primary Key)
- Name

Transaction

- **Transaction_Number** (Primary Key)
- Grade
- Offering_Number (Foreign Key)
- Student_Number (Foreign Key)

Relationships

Course_Prerequisite

- Between **Course** and **Course**

- Cardinality: **1 : M**
- Each course has at most one prerequisite.

Course_Offering

- Between **Course** and **Offering**
- Cardinality: **1 : M**

Offer_Teacher

- Between **Teacher** and **Offering**
- Cardinality: **1 : M**

Student_Transaction

- Between **Student** and **Transaction**
- Cardinality: **1 : M**

Offering_Transaction

- Between **Offering** and **Transaction**
- Cardinality: **1 : M**

Note: Check if repetition is needed or not for any M:M relationship

Question8: A database is to contain information about the internal education system of a large industrial company. The company in question maintains an education department whose function is to run several training courses for the employees of the company; each course is offered at a number of different locations within the organization, and the database contains details both of offerings already given and of offerings scheduled to be given in future. The database contains the following information:- For each course: course number (unique), course title, details of all immediate prerequisite course, and details of all offerings For each offering, of a given course; offering number, date, location, details of the teacher, and details of all students For each teacher of a given offering; employee number (unique), and name For each

student of a given offering; employee number (unique), name, and grade The following constraints apply:

- Each course has more than one prerequisite course, while one prerequisite course may be associated with many courses.

-Each course is taught by only one teacher, while a teacher may teach more than one course.

-Each course may have multiple offerings, while each offering belongs to one course.

-Each offering may include many students, and each student may be enrolled in many offerings.

ER Modeling Solution – Internal Education System Database

Solution 1: Using Relationships Using M:M Relationships (Direct Modeling)

Note: if repetition is not needed for M:M relationship

Entity Types

Course

- **Course_Number** (Primary Key)
- Course_Title

Offering

- **Offering_Number** (Primary Key)
- Offering_Date
- Location
- **Course_Number** (Foreign Key)
- **Teacher_Number** (Foreign Key)

Teacher

- **Employee_Number** (Primary Key)
- Name

Student

- **Employee_Number** (Primary Key)
- Name

Relationships

Prerequisite

- Recursive relationship on **Course**
- Cardinality: **M : M**
- One course may be a prerequisite for many courses.
- Each course has **more than one immediate prerequisite**.

Has_Offering

- Between **Course** and **Offering**
- Cardinality: **1 : M**
- Each course has multiple offerings.
- Each offering belongs to exactly one course.

Teaches

- Between **Teacher** and **Offer**
- Cardinality: **1 : M**

Attends

- Between **Student** and **Offering**
- Cardinality: **M : M**
- Relationship Attribute:
    - Grade

Solution 2: Resolving M:M Relationships Using Transaction Entity

Note: if repetition is needed for M:M relationship

Entity Types

Course

- **Course_Number** (Primary Key)
- Course_Title

Offering

- **Offering_Number** (Primary Key)
- Offering_Date
- Location
- Course_Number (Foreign Key)
- Teacher_Number (Foreign Key)

Teacher

- **Employee_Number** (Primary Key)
- Name

Student

- **Employee_Number** (Primary Key)
- Name

Transaction1

- **Transaction_Number1**(Primary Key)
- Grade
- Offering_Number (Foreign Key)
- Student_Employee_Number (Foreign Key)

Transaction2

- **Transaction_Number2** (Primary Key)
- Main_courseNumber (Foreign Key)
- Prerequisite_courseNumber(Foreign Key)

Relationships

CoursePrerequisite_Transaction2

- Between **Course** and **Transaction2**
- Cardinality: **1 : M**

CourseMain_Transaction2

- Between **Course** and **Transaction2**
- Cardinality: **1 : M**

Course_Offering

- Between **Course** and **Offering**
- Cardinality: **1 : M**

Offer_Teacher

- Between **Teacher** and **Offering**
- Cardinality: **1 : M**

Student_Transaction1

- Between **Student** and **Transaction1**
- Cardinality: **1 : M**

Offering_Transaction1

- Between **Offering** and **Transaction1**
- Cardinality: **1 : M**

Note: Check if repetition is needed or not for any M:M relationship.

Question9: The COMPANY database keeps track of company's employees, departments, and projects: - The company is organized into departments. Each department has a name, a number, an employee who manages the department. We keep track of the start date when that employee started managing the department. A department may have several locations. - A department controls a number of projects, each of which has a name, a number, and a single location. - We store each employee's name, social number, address, salary, sex, and birth date. An employee is assigned to one department but may work on several projects, which are not necessarily controlled by the same department. We keep track of the number of hours per week that an employee works on each project. We also keep track of the direct supervisor of each employee. - We want to keep track of the dependents of each employee for insurance purposes. We keep each dependent's name, sex, birthdate, and relationship to the employee.

ER Modeling Solution – COMPANY Database

Solution 1: Direct M:M modeling

Note: if repetition is not needed for M:M relationship

Entity Types

Department

- **Department_Number** (Primary Key)
- Department_Name
- **Locations** (Multivalued Attribute)

Employee

- **Social_Number** (Primary Key)
- Employee_Name
- Address
- Salary
- Sex
- Birth_Date

Project

- **Project_Number** (Primary Key)
- Project_Name
- Location

Dependent

- **Dependent_Name** (Partial Key)
- Sex
- Birth_Date
- Relationship

Relationships

Works_For

- Between **Employee** and **Department**
- Cardinality: **M : 1**
- Each employee works for one department.
- Each department has many employees.

Manages

- Between **Employee** and **Department**
- Cardinality: **1 : 1**
- Attribute: Manager_Start_Date
- Each department has exactly one manager.
- An employee may manage at most one department.

Controls

- Between **Department** and **Project**
- Cardinality: **1 : M**
- Each department controls many projects.
- Each project is controlled by exactly one department.

Works_On

- Between **Employee** and **Project**
- Cardinality: **M : M**
- Attribute: Hours_Per_Week
- An employee may work on multiple projects, not necessarily controlled by the same department.

Supervises

- Recursive relationship on **Employee**
- Cardinality: **1 : M**
- Each employee has one direct supervisor.
- A supervisor may supervise many employees.

Has_Dependent

- Between **Employee** and **Dependent**
- Cardinality: **1 : M**

Solution 2: Resolving M:M relationship using Transaction

Note: if repetition is needed for M:M relationship

Entity Types

Entity Types

Department

- **Department_Number** (Primary Key)
- Department_Name
- **Locations** (Multivalued Attribute)

Employee

- **Social_Number** (Primary Key)
- Employee_Name
- Address
- Salary

- Sex
- Birth_Date
- Supervisor_SSN (FK → Employee)

Project

- **Project_Number** (Primary Key)
- Project_Name
- Location
- Department_Number (FK → Department)

Dependent

- **Dependent_Name**
- Sex
- Birth_Date
- Relationship
- Social_Number (FK → Employee)
- **Primary Key:** (Dependent_Name, Social_Number)

Transaction

- **Transaction_Number** (Primary Key)
- Hours_Per_Week
- Social_Number (FK → Employee)
- Project_Number (FK → Project)

Relationships

Works_For

- Between **Employee** and **Department**

- Cardinality: **M : 1**
- Each employee works for exactly one department.
- Each department has many employees.

Manages

- Between **Employee** and **Department**
- Cardinality: **1 : 1**
- Attribute: Manager_Start_Date
- Each department has exactly one manager.

Controls

- Between **Department** and **Project**
- Cardinality: **1 : M**
- Each department controls multiple projects.

Employee_Transaction

- Between **Employee** and **Transaction**
- Cardinality: **1 : M**

Project_Transaction

- Between **Project** and **Transaction**
- Cardinality: **1 : M**

Supervises

- Recursive relationship on **Employee**
- Cardinality: **1 : M**

Employee_Dependent

- Between **Employee** and **Dependent**
- Cardinality: **1 : M**

Note: Check if repetition is needed or not for any M:M relationship

Question10: Hotel Booking and Reservation , for managing hotel room bookings, guests, ,room details and Staff: Rooms have a room number, type (single, double, suite), bed type, and price per night. Guests have a name, email, phone number, and check-in/check-out dates. Bookings are made by guests for specific rooms with a booking date and total cost. Staff manages bookings and can assist with special requests, each staff member has id,name,rank.

Scenario ER Solution – Hotel Booking & Reservation

Solution 1: Using M:M relationship (Direct)

Note: if repetition is not needed for M:M relationship

Entity Types

1. **Room**
    a. Room_Number (PK)
    b. Room_Type (Single, Double, Suite)
    c. Bed_Type
    d. Price_Per_Night
2. **Guest**
    a. Guest_ID (PK)
    b. Name
    c. Email
    d. Phone_Number
3. **Staff**
    a. Staff_ID (PK)
    b. Name
    c. Rank
4. **Booking**

a. Booking_ID (PK)
b. Booking_Date
c. Check_In_Date
d. Check_Out_Date
e. Total_Cost

Relationships

1. **Guest** ↔ **Booking** → 1 : M
    a. Each guest can make many bookings.
    b. Each booking belongs to exactly one guest.
2. **Staff** ↔ **Booking** → 1 : M
    a. Each staff member can manage multiple bookings.
    b. Each booking is managed by one staff member.
3. **Room** ↔ **Booking** → M : M
    a. A room can be booked in many bookings over time.
    b. A booking can include multiple rooms (for example, group bookings).
    c. Relationship Attributes: none (can add if needed: e.g., room price per booking).

Solution 2: Resolving M:M with Transaction

Note: if repetition is needed for M:M relationship

Entity Types

1. **Room**
    a. Room_Number (PK)
    b. Room_Type
    c. Bed_Type
    d. Price_Per_Night
2. **Guest**
    a. Guest_ID (PK)
    b. Name
    c. Email
    d. Phone_Number
3. **Staff**

    a. Staff_ID (PK)
    b. Name
    c. Rank
4. **Booking**
    a. Booking_ID (PK)
    b. Booking_Date
    c. Check_In_Date
    d. Check_Out_Date
    e. Total_Cost
    f. Guest_ID (FK → Guest)
    g. Staff_ID (FK → Staff)
5. **Transaction**
    a. Transaction_Number (PK)
    b. Booking_ID (FK → Booking)
    c. Room_Number (FK → Room)

Relationships

1. **Makes_Booking** → Guest ↔ Booking → 1 : M
    a. Each guest can make many bookings.
    b. Each booking belongs to exactly one guest.
2. **Manages_Booking** → Staff ↔ Booking → 1 : M
    a. Each staff member manages multiple bookings.
    b. Each booking is managed by one staff member.
3. **Includes_Room** → Booking ↔ Transaction → 1 : M
    a. Each booking can include multiple transactions (one per room).
    b. Each transaction belongs to exactly one booking.
4. **Room_Assignment** → Room ↔ Transaction → 1 : M
    a. Each room can appear in many transactions over time.
    b. Each transaction refers to exactly one room.

Note: Check if repetition is needed or not for any M:M relationship.

Question11: Transportation Fleet Management System for managing a transportation fleet, including vehicles, drivers, routes, and schedules: Vehicles have a license plate, type (truck, van, bus), capacity, and maintenance ,each vehicle is assigned to one driver and each maintenance has id,time,date,description details. Drivers have a name, license number, contact details, and assigned vehicle. Routes include a route ID, start

and end locations, and distance. Schedules specify the date and time when vehicles are assigned to routes to drivers.

Solution :

Entity Types and Attributes

1. **Vehicle**
    a. License_Plate (PK)
    b. Vehicle_Type (Truck, Van, Bus)
    c. Capacity
    d. Maintenance_Status
2. **Driver**
    a. Driver_ID (PK)
    b. Name
    c. License_Number
    d. Contact_Details
3. **Route**
    a. Route_ID (PK)
    b. Start_Location
    c. End_Location
    d. Distance
4. **Schedule**
    a. Schedule_ID (PK)
    b. Date
    c. Time
    d. Vehicle_ID (FK → Vehicle)
    e. Driver_ID (FK → Driver)
    f. Route_ID (FK → Route)
5. **Maintenance**
    a. Maintenance_ID (PK)
    b. Vehicle_ID (FK → Vehicle)
    c. Time
    d. Date
    e. Description

Relationships

1. **Responsible_For** → Vehicle ↔ Driver → 1 : 1

    a. Each vehicle is assigned to exactly one driver.
    b. Each driver is assigned to exactly one vehicle.
2. **Driver_Schedule** → Driver ↔ Schedule → 1 : M
    a. Each driver can have many schedules.
    b. Each schedule is assigned to exactly one driver.
3. **Vehicle_Schedule** → Vehicle ↔ Schedule → 1 : M
    a. Each vehicle can have many schedules.
    b. Each schedule uses exactly one vehicle.
4. **Route_Schedule** → Route ↔ Schedule → 1 : M
    a. Each route can have many schedules.
    b. Each schedule corresponds to exactly one route.
5. **Vehicle_Maintenance** → Vehicle ↔ Maintenance → 1 : M
    a. Each vehicle can have many maintenance records.
    b. Each maintenance record belongs to exactly one vehicle.

Question12: Restaurant Reservation and Ordering System Design a schema for managing restaurant reservations, orders, dishes, and staff: Dishes have a name, description, price, and ingredients. Customers make reservations with a date, time, and number of people. Orders are made by customers during their visit, with a list of dishes. Staff members handle orders and assist customers.

Solution 1: Direct M:M:M relationship

Note: if repetition is not needed for M:M relationship

Entity Types and Attributes

1. **Dish**
    a. Dish_ID (PK)
    b. Name
    c. Description
    d. Price
    e. **Ingredients** (Multivalued)
2. **Reservation**
    a. Reservation_ID (PK)
    b. Date
    c. Time

    d. Number_of_People
3. **Customer**
    a. Customer_ID (PK)
    b. Name
    c. Contact_Details
4. **Staff**
    a. Staff_ID (PK)
    b. Name
    c. Role

Relationships (Named)

1. **Reservation_Dishes** → Reservation ↔ Dish → 1 : M
    a. Each reservation can include multiple dishes.
    b. Each dish can appear in many reservations.
2. **Customer_Reservation** → Customer ↔ Reservation → 1 : M
    a. Each customer can make multiple reservations.
    b. Each reservation belongs to one customer.
3. **Customer_Reservation_Staff** → Customer ↔ Reservation ↔ Staff → M : M : M
    a. Represents staff assisting customers for each reservation.
    b. Each staff member can assist multiple customers across multiple reservations.

Solution 2: Resolving M:M:M using Transaction

Note: if repetition is needed for M:M relationship

Entity Types and Attributes

1. **Dish**
    a. Dish_ID (PK)
    b. Name
    c. Description
    d. Price
    e. **Ingredients** (Multivalued)

2. **Reservation**
    a. Reservation_ID (PK)
    b. Date
    c. Time
    d. Number_of_People
    e. Customer_ID (FK → Customer)
3. **Customer**
    a. Customer_ID (PK)
    b. Name
    c. Contact_Details
4. **Staff**
    a. Staff_ID (PK)
    b. Name
    c. Role
5. **Transaction**
    a. Transaction_Number (PK)
    b. Reservation_ID (FK → Reservation)
    c. Staff_ID (FK → Staff)
    d. Dish_ID (FK → Dish)

Relationships

1. **Customer_Reservation** → Customer ↔ Reservation → 1 : M
2. **Reservation_Transaction** → Reservation ↔ Transaction → 1 : M
3. **Staff_Transaction** → Staff ↔ Transaction → 1 : M
4. **Dish_Transaction** → Dish ↔ Transaction → 1 : M

Note: Check if repetition is needed or not for any M:M relationship.

Question13: A hospital has many registered physicians. Attributes of physicians include an id number and specialty. Patients are admitted to the hospital by physicians. Attributes of patients include a patient's id and name. Any patient who is admitted must have exactly one admitting physician. A physician may admit any number of patients and A patient may visit the hospital many times on different dates, registering for different treatments. Once admitted, a given patient must be treated by at least one physician. Some physicians may treat the same patient on different dates for different health issues. A particular treatment involves multiple physicians, and multiple patients may participate in the same treatment session or service. The service for any patient may be offered by multiple physicians across different dates-each recording the relevant

treatment and advice given. Also recording examination date within admitting and discharge dates.

Solution 1: Direct M:M relationship

Note: if repetition is not needed for M:M relationship

Entity Types and Attributes

1. **Physician**
    a. Physician_ID (PK)
    b. Specialty
2. **Patient**
    a. Patient_ID (PK)
    b. Name
3. **Treatment**
    a. Treatment_ID (PK)
    b. Date
    c. Health_Issue
    d. Advice_Given

Relationships

1. **Admits** → Physician ↔ Patient → M: M
    a. Each patient is admitted by exactly many physician.
    b. Each physician can admit many patients.
    c. Attributes on this relation (admit date, discharge date)
2. **Treats** → Physician ↔ Treatment ↔Patient→ M : M :M
    a. Each physician may treat multiple patients via treatments.
    b. Each treatment may involve multiple physicians.
    c. Attribute on relation(examination date)

Solution 2: Resolving M:M using Transaction

Note: if repetition is needed for M:M relationship

Entity Types and Attributes

1. **Physician**
    a. Physician_ID (PK)

   b. Specialty
2. **Patient**
   a. Patient_ID (PK)
   b. Name
3. **Treatment**
   a. Treatment_ID (PK)
   b. Date
   c. Health_Issue
   d. Advice_Given
4. **Transaction1**
   a. Transaction_Number1 (PK)
   b. Physician_ID (FK → Physician)
   c. Patient_ID (FK → Patient)
   d. admit date, discharge date

5. **Transaction2**

   a. Transaction_Number2 (PK)
   b. Transaction_Number1 (FK)

   c. Physician_ID (FK → Physician)
   d. Treatement_ID (FK → Patient)
   e. Examination_date

Relationships

1. **Physician_Transaction1** → Physician ↔ Transaction → 1 : M
   a. Each physician can participate in multiple transactions.
   b. Each transaction is handled by exactly one physician.
2. **Patient_Transaction1** → Patient ↔ Transaction → 1 : M
   a. Each patient can have multiple transactions.
   b. Each transaction involves exactly one patient.

3. **Physician _Transaction2** → Physician ↔ Transaction2 → 1 : M

   a. Each Physician can have multiple transactions.
   b. Each transaction involves exactly one Physician.

4. **Treatment_Transaction2** → Treatment↔ Transaction2 → 1 : M

    a. Each Treatment can have multiple transactions.
    b. Each transaction involves exactly one Treatment.

5. **Transaction1_Transaction2** → Transaction1↔ Transaction2 → 1 : M

    c. Each Transaction1 can have multiple transaction2.
    d. Each transaction involves exactly one Transaction1.

Note: Check if repetition is needed or not for any M:M relationship.

Question14: In a mini world of a university, each semester, each student must be assigned an advisor who advises students about degree requirements and helps students register for classes. Each student must register for classes with the help of an advisor, but if the student's assigned advisor is not available, then the student may register with any advisor. We want to keep track of: • Students • The assigned advisor for each student • The advisor with whom the student registered for the current term.

Solution 1: Direct M:N relationships

Note: if repetition is not needed for M:N relationship

Entity Types and Attributes

1. **STUDENT**
    a. Student_ID (PK)
    b. Name
2. **ADVISOR**
    a. Advisor_ID (PK)
    b. Name
3. **SEMESTER**
    a. Semester_NAME (PK)
    b. Year
4. **CLASS**
    a. C_ID (PK)
    b. C_NAME

Relationships

1. **Is_Assigned_To** → ADVISOR ↔ STUDENT → 1 : M
    a. Each student is assigned exactly one advisor.
    b. Each advisor can advise multiple students.
2. **Registers_With** → STUDENT ↔ CLASS → M : N
    a. Modeled by **REGISTRATION** entity;
    b. Attributes: Student_ID (FK), C_ID (FK), Semester_NAME (FK)
    c. PK = Student_ID + C_ID
3. **Includes** → SEMESTER ↔ ADVISOR → M : N
    a. Modeled by **INCLUDES** entity.
    b. Attributes: Advisor_ID (FK), Semester_NAME (FK)
    c. PK = Advisor_ID + Semester_NAME
4. **Join** → SEMESTER ↔ CLASS → M : N
    a. Modeled by **SEMESTER_JOIN_CLASS** entity.
    b. Attributes: C_ID (FK), Semester_NAME (FK)
    c. PK = C_ID + Semester_NAME

Solution 2: Resolving M:N relationships using Transaction

Note: if repetition is needed for M:N relationship

Entity Types and Attributes

1. **STUDENT**
    a. Student_ID (PK)
    b. Name
2. **ADVISOR**
    a. Advisor_ID (PK)
    b. Name
3. **SEMESTER**
    a. Semester_NAME (PK)
    b. Year
4. **CLASS**
    a. C_ID (PK)
    b. C_NAME
5. **Transaction**

a. Transaction_Number (PK)
b. Student_ID (FK → STUDENT)
c. Advisor_ID (FK → ADVISOR)
d. Semester_NAME (FK → SEMESTER)
e. C_ID (FK → CLASS)

Relationships

1. **Is_Assigned_To** → ADVISOR ↔ STUDENT → 1 : M
2. **Transaction_Student** → STUDENT ↔ Transaction → 1 : M
    a. Each student can have multiple transactions.
3. **Transaction_Advisor** → ADVISOR ↔ Transaction → 1 : M
    a. Each advisor can participate in multiple transactions.
4. **Transaction_Semester** → SEMESTER ↔ Transaction → 1 : M
    a. Each semester can have multiple transactions.
5. **Transaction_Class** → CLASS ↔ Transaction → 1 : M
    a. Each class can appear in multiple transactions.

Note: Check if repetition is needed or not for any M:M relationship.

Question15: Customers of a bank are identified by a unique customer ID and have a name and address. They can hold accounts, each identified by a unique account number and current balance. Customers can also gain loans, with each loan having a unique loan ID and amount. Bank branches are identified by a unique branch code and have a location. Each branch can offer multiple types of loans.

Solution: Entity Types and Attributes

1. **CUSTOMER**
    a. Customer_ID (PK)
    b. Name
    c. Address
2. **ACCOUNT**
    a. Account_Number (PK)
    b. Current_Balance
    c. Branch_ID (FK → BANK)
3. **LOAN**
    a. Loan_ID (PK)
    b. Amount
    c. Loan_Type_ID (FK → LOAN_TYPE)

    d. Customer_ID (FK → CUSTOMER)
4. **BANK**
    a. Branch_ID (PK)
    b. Location
5. **LOAN_TYPE**
    a. Loan_Type_ID (PK)
    b. Type_Name
    c. Branch_ID (FK → BANK) ← **to model 1:M "Offers" relationship**

Relationships and Cardinalities (All 1:M)

1. **Holds** → CUSTOMER (1) : ACCOUNT (M)
    a. Each customer can hold multiple accounts.
    b. Each account is held by exactly one customer.
2. **Borrows** → CUSTOMER (1) : LOAN (M)
    a. Each customer can have multiple loans.
    b. Each loan is taken by exactly one customer.
3. **Offers** → BANK (1) : LOAN_TYPE (M)
    a. Each bank branch can offer multiple loan types.
    b. Each loan type is offered by exactly one bank branch.
4. **Located_At** → BANK (1) : ACCOUNT (M)
    a. Each account is created at exactly one bank branch.
    b. Each bank branch can have multiple accounts.
5. **Is_Of_Type** → LOAN_TYPE (1) : LOAN (M)
    a. Each loan is of exactly one loan type.
    b. Each loan type can be associated with multiple loans.

Question 16: An airline company has passengers, each identified with a unique passenger ID, along with their name and passport number. Flights are identified by a unique flight number and have a departure and arrival time. Each flight is operated by an airplane, which is identified by a unique airplane ID, make, and model. Passengersbook tickets for these flights, and each ticket has a unique ticket number.

Solution: Entity Types and Attributes

1. **PASSENGER**
    a. Passenger_ID (PK)
    b. Name
    c. Address

2. **FLIGHT**
    a. Flight_ID (PK)
    b. Departure_Time
    c. Arrival_Time
3. **AIRPLANE**
    a. Airplane_ID (PK)
    b. A-TYPE
4. **TICKET**
    a. Ticket_ID (PK)
    b. Price

Relationships and Cardinalities

1. **Books** → PASSENGER (1) : TICKET (M)
    a. One passenger can book many tickets.
    b. Each ticket is booked by exactly one passenger.
2. **Is_For** → FLIGHT (1) : TICKET (M)
    a. Each ticket is for one flight.
    b. One flight can have many tickets.
3. **Operated_By** → AIRPLANE (1) : FLIGHT (M)
    a. Each flight is operated by exactly one airplane.
    b. One airplane can operate many flights.

Question 17: The Library Management System database keeps track of readers with the following considerations:

The system keeps track of the staff with Staff name and staff id also with a single-point authentication system (login ID and password).

Staff maintains the book with its ISBN, title, price, category, number of copies, edition, publisher number, and book name , and for each maintenance activity , the maintenance date and type are recorded.

A publisher has a publisher ID, year of publication, and book name.

Readers are registered with the following attributes: user_id, email, name (first name and last name), phone number(s), and communication address.

Readers can borrow, return, or reserve books. For each borrowing transaction, the system records the borrow date, due date, and actual return date.

ER Modeling Solution – Library Management System Database

Solution 1: Using M:M Relationships (Direct Modeling)

Note: This solution is used when repetition is *not* required for the M:M relationship.

Entity Types

Staff

- Staff_ID (Primary Key)
- Staff_Name
- Login_ID
- Password

Reader

- User_ID (Primary Key)
- Email
- First_Name
- Last_Name
- Phone_Number
- Address

Book

- ISBN (Primary Key)
- Title
- Price
- Category
- Number_of_Copies
- Edition
- publisher_ID(Foreign key)
- Book_Name

Publisher

- Publisher_ID (Primary Key)
- Year_of_Publication

Relationships

Publishes

Between: Publisher and Book
Cardinality: 1 : M

- One publisher may publish many books.
- Each book is published by only one publisher.

Maintains

Between: Staff and Book
Cardinality: M : M

- One staff member may maintain many books.
- Each book is maintained by more than one staff member.

Relationship Attributes:

- Maintenance_date
- Maintenance_type

Borrows

Between: Reader and Book
Cardinality: M : M

Relationship Attributes:

- Borrow_Date
- Due_Date
- Actual_Return_Date

Solution 2: Resolving M:M Relationships Using a Transaction Entity

Note: This solution is used when repetition *is required* for the M:M relationship.

Entity Types

Staff

- Staff_ID (Primary Key)
- Staff_Name
- Login_ID
- Password

Reader

- User_ID (Primary Key)
- Email
- First_Name
- Last_Name
- Phone_Number
- Address

Book

- ISBN (Primary Key)
- Title
- Price
- Category
- Number_of_Copies
- Edition
- Author_Number
- Book_Name
- Publisher_ID (Foreign Key)

Publisher

- Publisher_ID (Primary Key)
- Year_of_Publication

Borrowing_Transaction

- Transaction_ID (Primary Key)
- Borrow_Date
- Due_Date
- Actual_Return_Date
- Reader_ID (Foreign Key → Reader)
- ISBN (Foreign Key → Book)

Maintenace_Transaction

- Transaction_ID2 (Primary Key)
- Maintenace_date
- Maintenace_type
- Staff_ID (Foreign Key → Staff)
- ISBN (Foreign Key → Book)

Relationships

Publisher_Book

Between: Publisher and Book
Cardinality: 1 : M

Staff_ Maintenace_Transaction

Between: Staff and Maintenace_Transaction
Cardinality: 1 : M

Book_ Maintenace_Transaction

Between: Book and Maintenace_Transaction
Cardinality: 1 : M

Reader_ Borrowing_Transaction

Between: Reader and Borrowing_Transaction
Cardinality: 1 : M

Book_ Borrowing_Transaction

Between: Book and Borrowing_Transaction
Cardinality: 1 : M

Note: Check if repetition is needed or not for any M:M relationship